

pavanpw-36

Reporters

we can change the report we want either in pw config or in cmd line.

built in reporters-

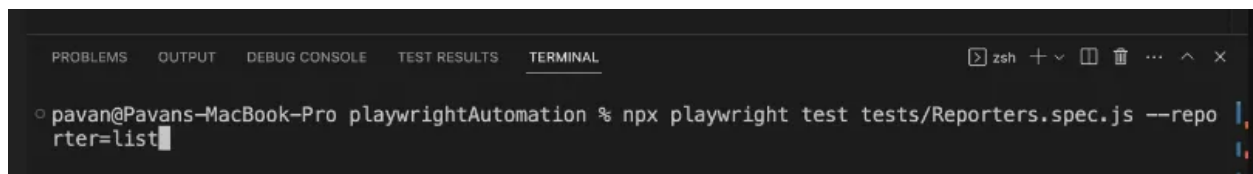
list reporter-



```
playwright.config.js  playwrightAutomation
playwright.config.js x
playwright.config.js > [unknown] > reporter
20   retries: process.env.CI ? 2 : 0,
21   /* Opt out of parallel tests on CI. */
22   workers: process.env.CI ? 2 : undefined,
23   /* Reporter to use. See https://playwright.dev/docs/test-reporters */
24   reporter: 'list',
25
```

set it directly without pw config on cmd-

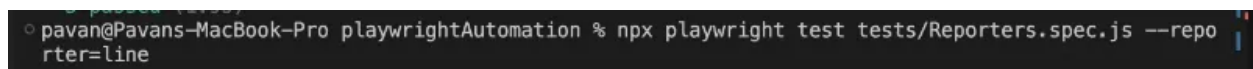
tick marks in report.



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TEST RESULTS  TERMINAL
pavan@Pavans-MacBook-Pro playwrightAutomation % npx playwright test tests/Reporters.spec.js --reporter=list
```

line reporter-

nothing in reports.



```
pavan@Pavans-MacBook-Pro playwrightAutomation % npx playwright test tests/Reporters.spec.js --reporter=line
```

dot reporter-

..F

F means fail

.. means pass

```
o pavan@Pavans-MacBook-Pro playwrightAutomation % npx playwright test tests/Reporters.spec.js --reporter=dot
```

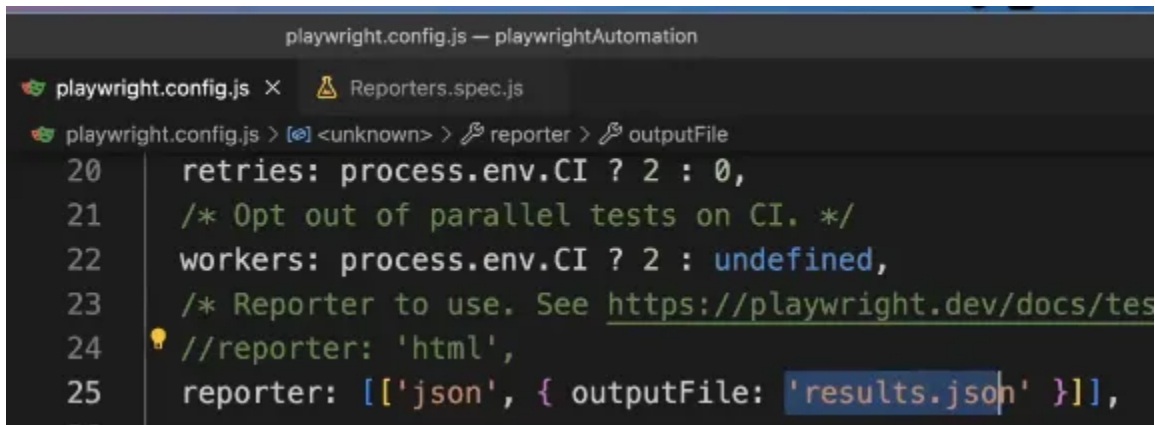
html reporter-

default report which we get, see it either in pw report folder or using cmd.

```
o pavan@Pavans-MacBook-Pro playwrightAutomation % npx playwright test tests/Reporters.spec.js --reporter=html
```

json reporter-

slightly different in config file.



```
playwright.config.js — playwrightAutomation
playwright.config.js x Reporters.spec.js
playwright.config.js > <unknown> > reporter > outputFile
20   retries: process.env.CI ? 2 : 0,
21   /* Opt out of parallel tests on CI. */
22   workers: process.env.CI ? 2 : undefined,
23   /* Reporter to use. See https://playwright.dev/docs/test-reports */
24   //reporter: 'html',
25   reporter: [['json', { outputFile: 'results.json' }]],
```

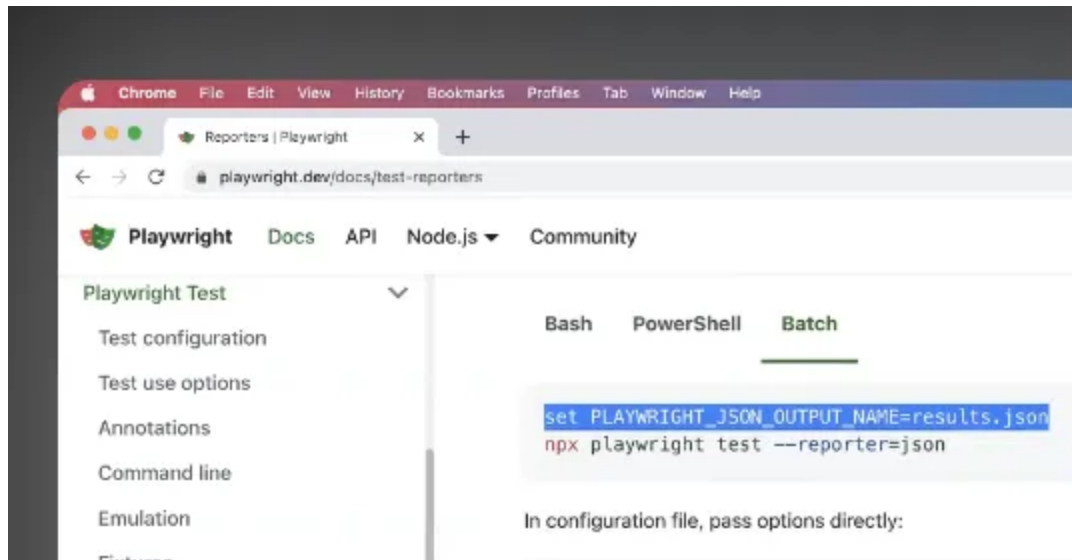
output is created at project root level.

will generate json in command line itself as we didnt specify path.-

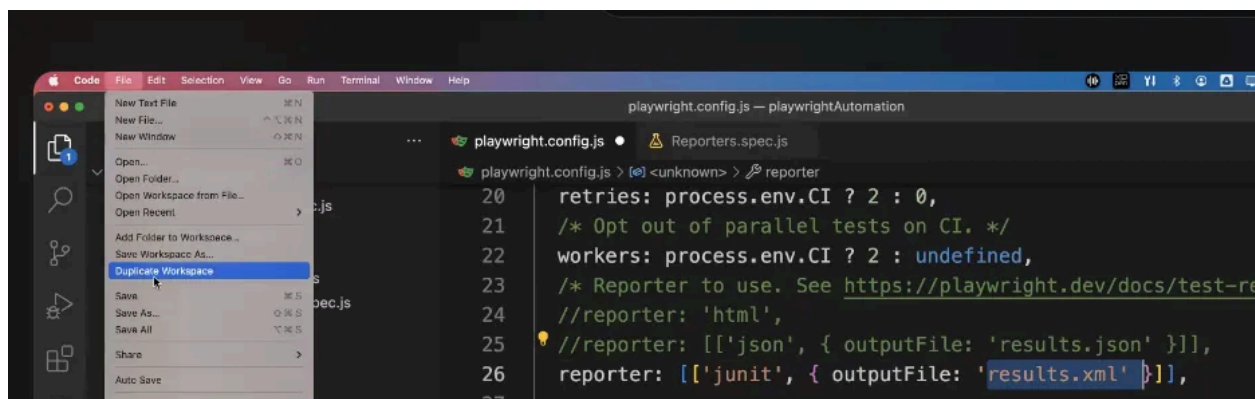
```
o pavan@Pavans-MacBook-Pro playwrightAutomation % npx playwright test tests/Reporters.spec.js --reporter=json
```

to generate in file-

first we need to run bash command and then the report generation command.

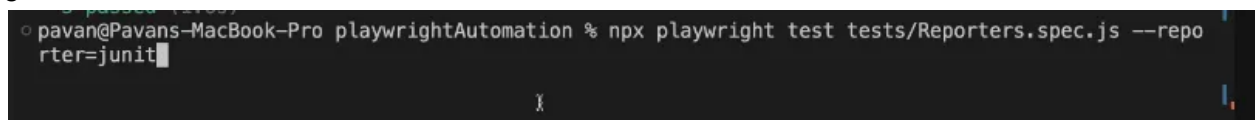


junit reporter-

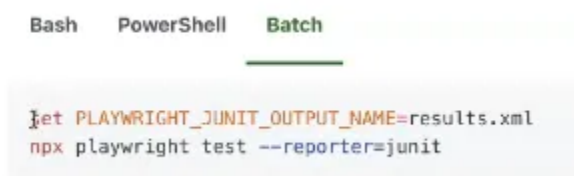


instead of json, xml is generated.

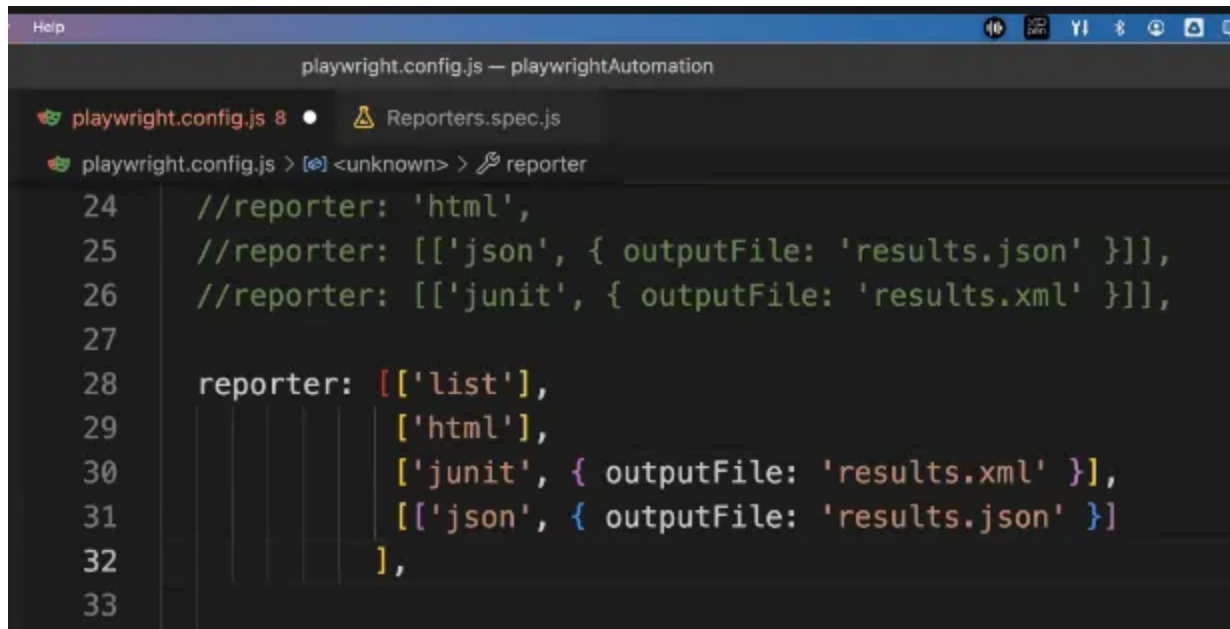
generated in console.-



set the bash command for specific location-

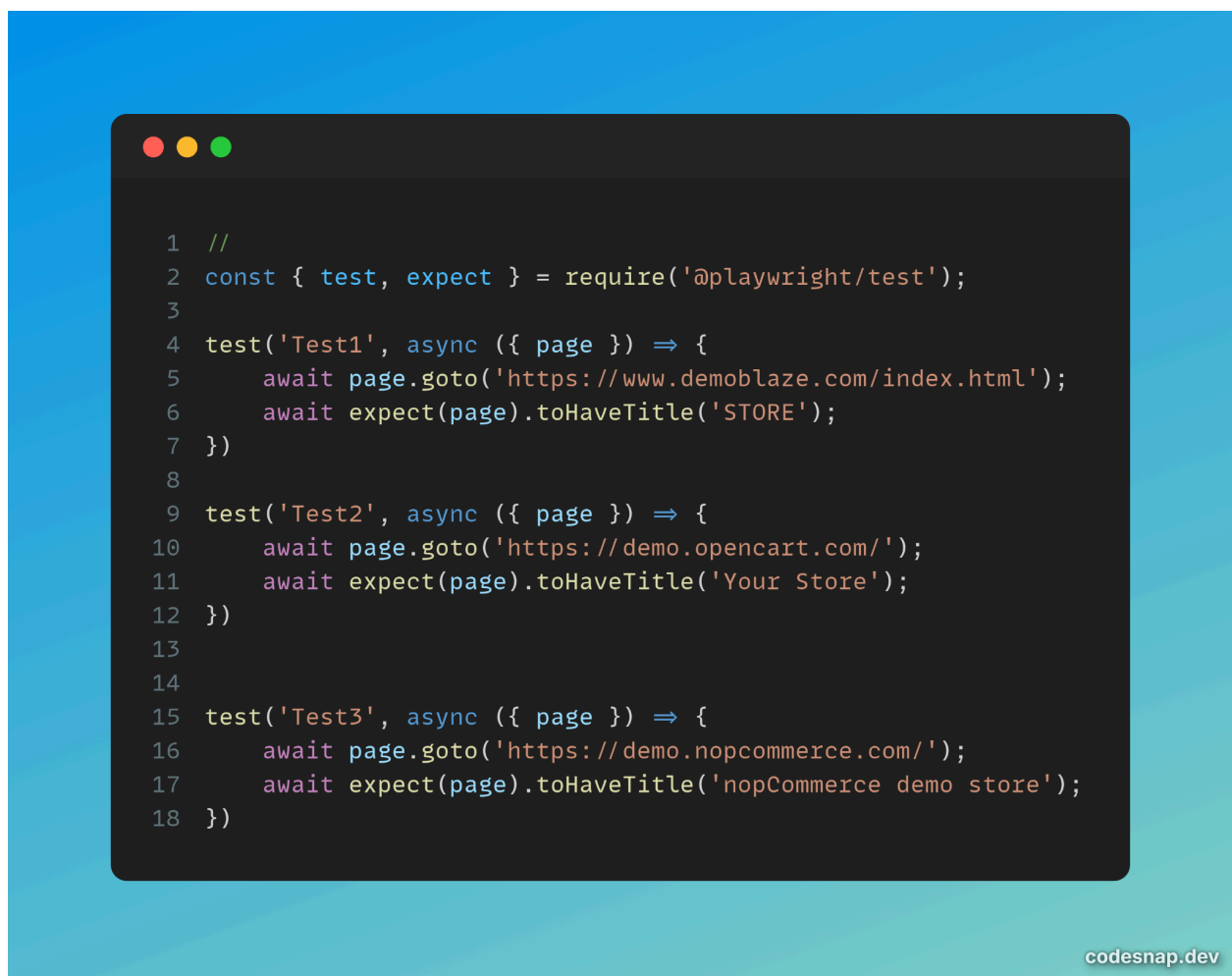


add multiple reporters-



A screenshot of the Visual Studio Code editor interface. The top bar shows the title 'playwright.config.js — playwrightAutomation'. Below the title bar, there are two tabs: 'playwright.config.js 8' and 'Reporters.spec.js'. The 'playwright.config.js' tab is active, and the cursor is positioned at line 24. The code in the editor is as follows:

```
24 //reporter: 'html',
25 //reporter: [['json', { outputFile: 'results.json' }]],
26 //reporter: [['junit', { outputFile: 'results.xml' }]],
27
28 reporter: [['list',
29            ['html',
30             ['junit', { outputFile: 'results.xml' }],
31             ['json', { outputFile: 'results.json' }]]],
32            ],
33
```



A screenshot of a code editor window with a blue background. The editor contains a JavaScript file with Playwright test code. The code is as follows:

```
1 //
2 const { test, expect } = require('@playwright/test');
3
4 test('Test1', async ({ page }) => {
5     await page.goto('https://www.demoblaze.com/index.html');
6     await expect(page).toHaveTitle('STORE');
7 })
8
9 test('Test2', async ({ page }) => {
10    await page.goto('https://demo.opencart.com/');
11    await expect(page).toHaveTitle('Your Store');
12 })
13
14
15 test('Test3', async ({ page }) => {
16    await page.goto('https://demo.nopcommerce.com/');
17    await expect(page).toHaveTitle('nopCommerce demo store');
18 })
```

codesnap.dev